Méthodologie

Étapes :

1. Analyse initiale et préparation p.1
2. Schéma relationnel normalisé p.2-4
3. Requêtage et résultats p.5-12

1. Analyse initiale et préparation

Cette étape a consisté à analyser la structure et les types de données des tables ***contrat*** et ***region*** au format csv.

L'objectif de cette analyse était de collecter les informations nécessaires pour alimenter et structurer **le dictionnaire des bases de données** (format xls) en y listant les entêtes de colonnes, le type de données et une description courte du contenu de chaque colonne.

Dictionnaire des données

	Nom des colonnes	Type de données	Taille	Clé	Description
CONTRAT.CSV	Contrat_ID	INT NOT NULL		Clé primaire	Id unique pour les contrats
	Code_dep_code_commune	CVARCHAR NOT NULL		Clé secondaire	Concaténation du code département et code commune pour avoir une clé unique
	No_voie	INT			Numéro dans la voie pour l'adresse du logement assuré
	Valeur_declaree_biens	CVARCHAR			Valeur déclarée du bien par tranches
	B_T_Q	CHAR	1		Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Prix_cotisation_mensuel	INT			Montant de la cotisation
	Type_de_voie	CVARCHAR			Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Occupation	CVARCHAR			Statut d'occupation : propriétaire, locataire.
	Type_contrat	CVARCHAR			Type de contrat : résidence principale, secondaire, etc.
	Formule	CVARCHAR			Formule choisie : intégrale, classique
	Voie	CVARCHAR			Libellé de la voie pour l'adresse du logement assuré
	Surface	INT			Surface totale du local en mètres carrés
	Type_local	CVARCHAR			Type de local : appartement, maison, etc.
	Code_postal	INT			Code postal pour l'adresse du logement assuré
REGION.CSV	Code_dep_code_commune	CVARCHAR NOT NULL		Clé primaire	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INT			Code numérique attribué à chaque région (exemple : 94 pour la Corse).
	reg_nom	CVARCHAR			Nom de la région (exemple : Corse).
	aca_nom	CVARCHAR			Nom de l'académie associée à la région (exemple : Corse).
	dep_nom	CVARCHAR			Nom du département (exemple : Haute-Corse).
	com_nom	CVARCHAR			Nom de la commune dans le département (exemple : AGHIONE, ALANDO).
	dep_code	CVARCHAR			Code à deux caractères représentant chaque département (exemple : 2B).
	dep_nom_num	CVARCHAR			Nom du département associé à son code, incluant le numéro (exemple : Haute-Corse (2B)).

Table ***contrat***

A	B	C	D	E	F	G	H	I	J	K	L	M	N
Contrat_ID	Code_dep_code_commune	No_voie	Valeur_declaree_biens	B_T_Q	Prix_cotisation_mensuel	Type_de_voie	Occupation	Type_contrat	Formule	Voie	Surface	Type_local	Code_postal
100601	1350	190	0-25000	A	25	RUE	Locataire	Residence princ	Integral	CENTRA	50	Appartement	1370
100602	1103	347	0-25000		30	RUE	Locataire	Residence princ	Classique	DU CHA	48	Appartement	1170
100603	1143	58	25000-50000		57	AV	Proprietaire	Residence princ	Integral	DU MON	131	Appartement	1220
100604	1288	140	25000-50000		43	RUE	Locataire	Residence princ	Integral	DE L'ABI	109	Maison	1630

Table ***region***

A	B	C	D	E	F	G	H
Code_dep_code_commune	reg_code	reg_nom	aca_nom	dep_nom	com_nom	dep_code	dep_nom_num
1001	84	Auvergne-Rhô	Lyon	Ain	L ABERGEMENT CLEMEI	1	Ain (01)
1002	84	Auvergne-Rhô	Lyon	Ain	L ABERGEMENT DE VAR	1	Ain (01)
1003	84	Auvergne-Rhô	Lyon	Ain	AMAREINS	1	Ain (01)

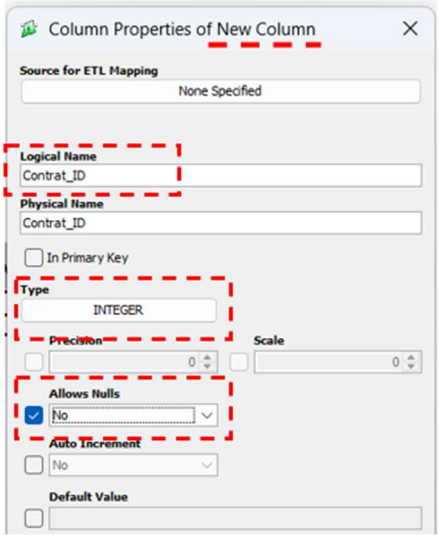
2. Schéma relationnel normalisé

Une fois le dictionnaire des bases de données renseigné, cette étape a consisté à créer et modéliser les tables ***contrat*** et ***region*** à l'aide du logiciel **SQL Power Architect**.

Voici les manipulations principales réalisés :

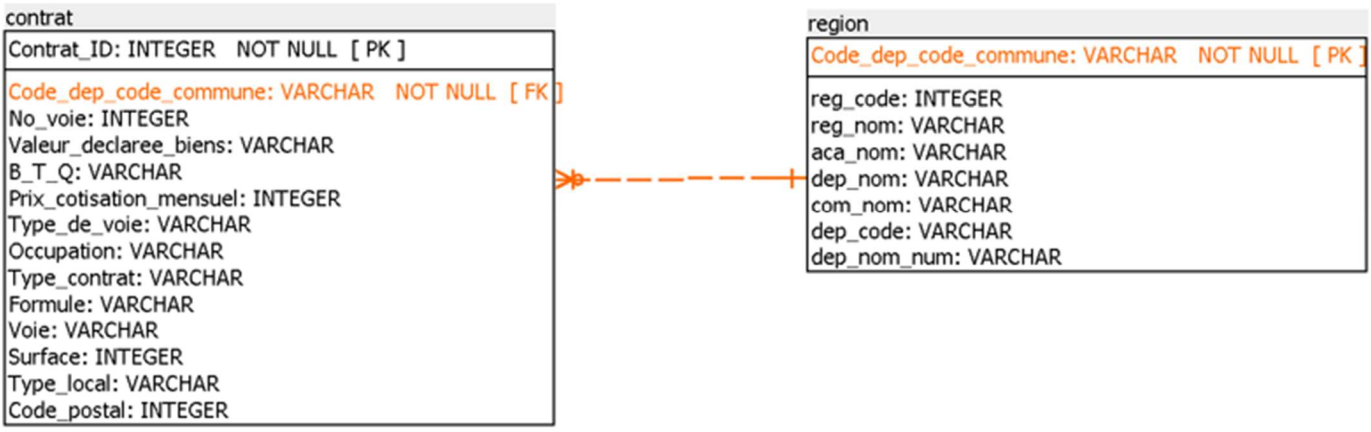
a) Création des tables : Les entêtes de colonnes ont été saisies à en respectant les informations du dictionnaire.

b) Définition des types de données : Chaque colonne a été configurée avec le type de données approprié. Certaines colonnes comme Contrat_ID ont été paramétrées avec des contraintes. Exemple de colonne avec la contrainte NOT NULL (les cellules vides ne sont pas acceptées) :



c) Configuration des clés :

- La clé primaire de la table **contrat** a été définie sur la colonne [Contrat_ID].
- La clé primaire de la table **region** a été définie sur la colonne [Code_dep_code_commune].
- La colonne [Code_dep_code_commune] de la table **contrat** a été définie comme clé étrangère afin de pouvoir relier les deux tables.



d) Générer le code SQL : Une fois le schéma relationnel finalisé, le logiciel SQL Power Architect a permis de générer le code SQL correspondant.

```
CREATE TABLE public.region (
	Code_dep_code_commune VARCHAR NOT NULL,
	reg_code INTEGER,
	reg_nom VARCHAR,
	aca_nom VARCHAR,
	dep_nom VARCHAR,
	com_nom VARCHAR,
	dep_code VARCHAR,
	dep_nom_num VARCHAR,
	CONSTRAINT code_dep_code_commune PRIMARY KEY (Code_dep_code_commune)
);
CREATE TABLE public.contrat (
	Contrat_ID INTEGER NOT NULL,
	Code_dep_code_commune VARCHAR NOT NULL,
	No_voie INTEGER,
	Valeur_declaree_biens VARCHAR,
	B_T_Q VARCHAR,
	Prix_cotisation_mensuel INTEGER,
	Type_de_voie VARCHAR,
	Occupation VARCHAR,
	Type_contrat VARCHAR,
	Formule VARCHAR,
	Voie VARCHAR,
	Surface INTEGER,
	Type_local VARCHAR,
	Code_postal INTEGER,
	CONSTRAINT contrat_id_pk PRIMARY KEY (Contrat_ID)
);
ALTER TABLE public.contrat ADD CONSTRAINT region_contrat_fk
FOREIGN KEY (Code_dep_code_commune)
REFERENCES public.region (Code_dep_code_commune)
ON DELETE NO ACTION
ON UPDATE NO ACTION
NOT DEFERRABLE;
```

e) Création de la base de données dans un SGBDR : Le code SQL généré sous SQL Power Architect a permis de créer la base de données dans pgAdmin qui est le système de gestion de bases de données relationnel que j'ai utilisé pour ce projet.

Table ***contrat***

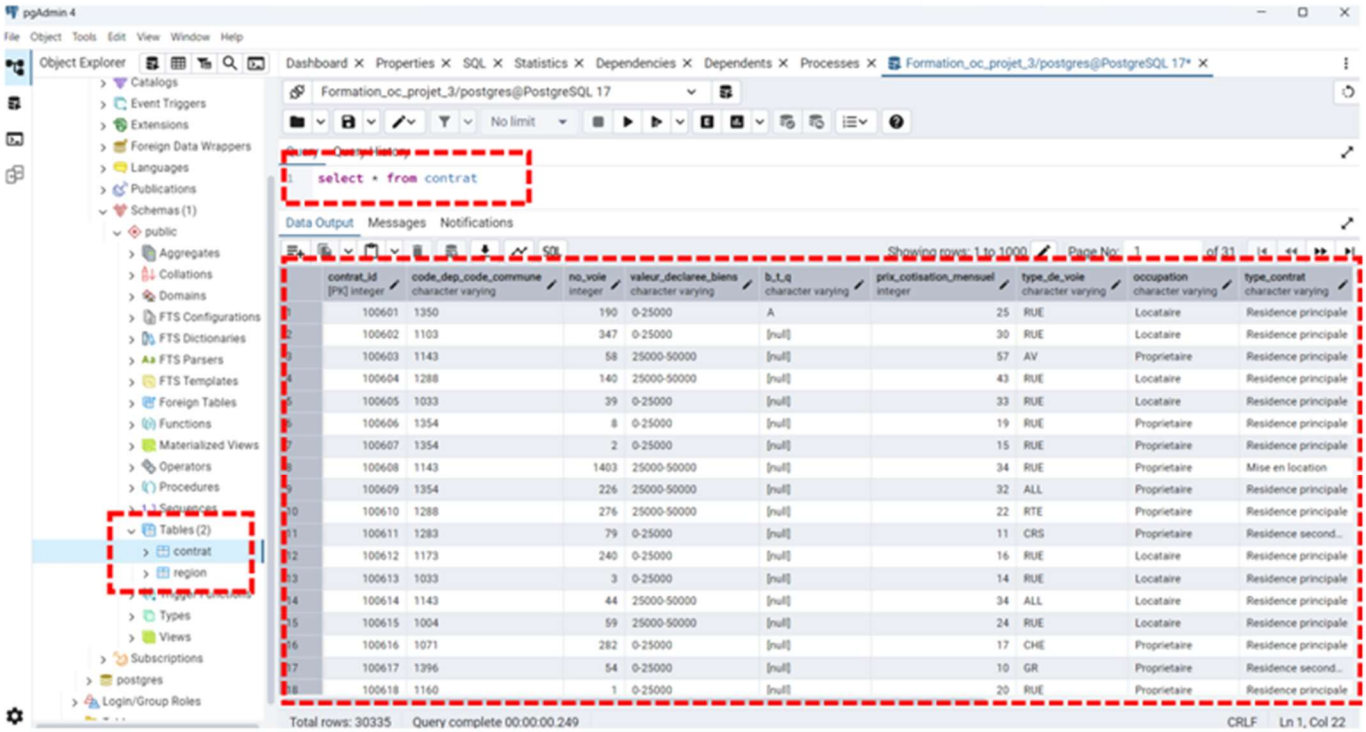
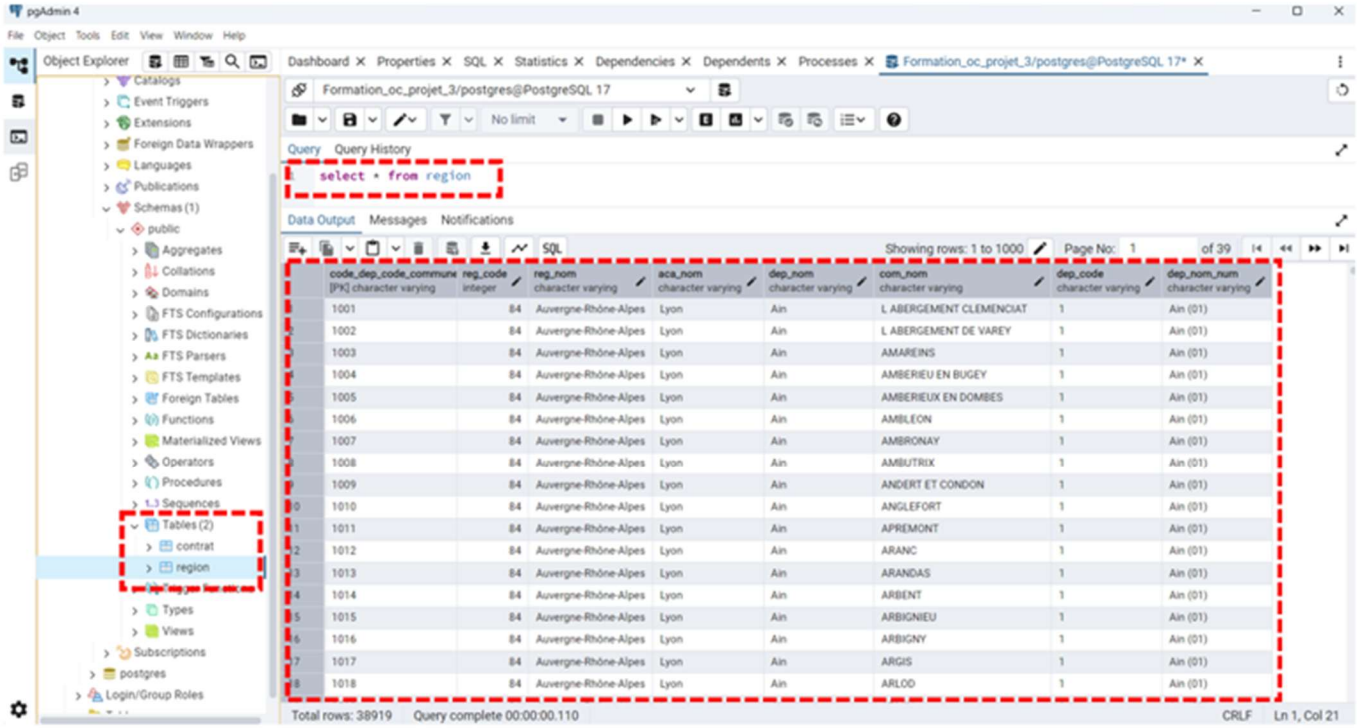


Table ***region***



3. Requêtage et résultats

La base de données étant créée, une série de 12 requêtes a été traitée :

Requête 1

Lister les numéros de contrats (contrat_ID) avec leur surface pour la commune de Caen.

Code SQL :

```
SELECT contrat.Contrat_ID, contrat.Surface
FROM contrat
JOIN region
ON contrat.Code_dep_code_commune = region.Code_dep_code_commune
WHERE region.com_nom = 'CAEN';
```

Résultat :

	contrat_id [PK] integer	surface integer
1	103791	35
2	103792	99
3	103793	40
4	103794	20

Description du code :

La fonction **SELECT** extrait les numéros de contrat [Contrat_ID] et les surfaces [Surface] de la table ***contrat***. La fonction **JOIN** relie cette table à ***region*** via la clé commune [Code_dep_code_commune]. La clause **WHERE** filtre les données pour inclure uniquement celles correspondant à la commune de Caen ([com_nom] = 'CAEN').

Requête 2

Lister les numéros de contrats avec le type de contrat et leur formule pour les maisons du département 71.

Code SQL :

```
SELECT c.Contrat_ID, c.Type_contrat, c.Formule
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune = r.Code_dep_code_commune
WHERE r.dep_code = '71' AND c.Type_local = 'Maison';
```

Résultat :

	contrat_id [PK] integer	type_contrat character varying	formule character varying
1	114768	Residence principale	Integral
2	114779	Residence principale	Classique
3	114782	Residence principale	Classique
4	114812	Residence principale	Integral

Description du code :

La fonction **SELECT** extrait les numéros de contrats [Contrat_ID], le type de contrat [Type_contrat] et la formule [Formule]. La fonction **JOIN** relie les tables ***contrat*** et ***region*** via la clé commune [Code_dep_code_commune]. La clause **WHERE** filtre les données pour inclure uniquement les maisons ([Type_local] = 'Maison') situées dans le département 71 ([dep_code] = '71').

Requête 3

Lister le nom des régions de France.

Code SQL :

```
SELECT DISTINCT reg_nom AS "Liste des régions"
FROM region;
```

Résultat :

	reg_nom character varying
1	Hauts-de-France
2	La Réunion
3	Bretagne
4	Mayotte
5	Auvergne-Rhône-Alpes
6	Ile-de-France
7	Grand Est
8	Corse
9	Collectivités d'outre-mer
10	Guyane
11	Nouvelle C
12	Bourgogne-Franche-Comté
13	Provence-Alpes-Côte d'Azur
14	Martinique
15	Guadeloupe
16	Normandie
17	Nouvelle-Aquitaine
18	Occitanie

Total rows: 20 Query complete

Description du code :

Avec la fonction **SELECT DISTINCT**, cette requête extrait les noms uniques des régions de France via la colonne [reg_nom]. La commande **AS** renomme les résultats affichés sous l'intitulé "Liste des régions" pour une présentation claire.

Requête 4

Quels sont les 5 contrats qui ont les surfaces les plus élevées ?

Code SQL :

```
SELECT Contrat_ID, Surface
FROM contrat
ORDER BY Surface DESC
LIMIT 5;
```

Résultat :

	contrat_id [PK] integer	surface integer
1	104211	815
2	105463	742
3	130878	595
4	100822	570
5	109872	559

Description du code :

La fonction **SELECT** extrait les numéros de contrat [Contrat_ID] et leurs surfaces [Surface]. La fonction **ORDER BY** trie les résultats par surface en ordre décroissant (DESC). La clause **LIMIT** restreint l'affichage aux 5 contrats avec les surfaces les plus élevées.

Requête 5

Quel est le prix moyen de la cotisation mensuelle ?

Code SQL :

```
SELECT ROUND (AVG (Prix_cotisation_mensuel),2)
AS "Prix moyen de la cotisation mensuelle"
FROM contrat;
```

Résultat :

	Prix moyen de la cotisation mensuelle numeric
1	19.33

Description du code :

La fonction **SELECT** calcule le prix moyen des cotisations mensuelles avec **AVG**, appliquée à la colonne [Prix_cotisation_mensuel] de la table ***contrat***. La fonction **ROUND** arrondit le résultat à deux décimales, et la commande **AS** renomme ce résultat en "Prix moyen de la cotisation mensuelle".

Requête 6

Quel est le nombre de contrats pour chaque catégorie de prix de la valeur déclarée des biens ?

Code SQL :

```
SELECT valeur_declaree_biens AS categorie_prix,
    COUNT(*) AS nombre_de_contrats
FROM contrat
GROUP BY valeur_declaree_biens
ORDER BY valeur_declaree_biens;
```

Résultat :

	categorie_prix character varying	nombre_de_contrats bigint
1	0-25000	22720
2	100000+	104
3	25000-50000	6815
4	50000-100000	696

Description du code :

La fonction **SELECT** extrait les catégories de prix depuis [valeur_declaree_biens] de la table ***contrat***. La fonction **COUNT** calcule le nombre de contrats pour chaque catégorie. La commande **GROUP BY** regroupe les données par catégories de prix, et la clause **ORDER BY** trie les résultats par ordre croissant des catégories ([valeur_declaree_biens]).

Requête 7

Quel est le nombre de formules “integral” sur la région Pays de la Loire ?

Code SQL :

```
SELECT COUNT(*) AS "Nombre de formules Integral dans le Pays de la Loire"
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune = r.Code_dep_code_commune
WHERE r.reg_nom = 'Pays de la Loire' AND c.formule = 'Integral';
```

Résultat :

	Nombre de formules Integral dans le Pays de la Loire bigint
1	589

Description du code :

La fonction **SELECT** avec **COUNT** dénombre les contrats ayant la formule [Integral] dans la colonne [formule] de la table ***contrat***. La fonction **JOIN** relie les tables ***contrat*** et ***region*** via la clé commune [Code_dep_code_commune]. La clause **WHERE** filtre les données pour inclure uniquement celles de la région Pays de la Loire ([reg_nom] = 'Pays de la Loire') avec la formule [Integral].

Requête 8

Lister les numéros de contrats avec le type de contrat et leur formule pour les maisons du département 71.

Code SQL :

```
SELECT c.Contrat_ID, c.Type_contrat, c.Formule
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune = r.Code_dep_code_commune
WHERE c.Type_local = 'Maison' AND r.dep_code = '71';
```

Résultat :

	contrat_id [PK] integer	type_contrat character varying	formule character varying
1	114768	Residence principale	Integral
2	114779	Residence principale	Classique
3	114782	Residence principale	Classique
4	114812	Residence principale	Integral

Description du code :

La fonction **SELECT** extrait le numéro de contrat [Contrat_ID], le type de contrat [Type_contrat] et la formule [Formule] depuis la table ***contrat***. La fonction **JOIN** relie cette table à ***region*** via la clé commune [Code_dep_code_commune]. La clause **WHERE** filtre les résultats pour les contrats concernant les maisons ([Type_local], table ***contrat***) situées dans le département 71 ([dep_code], table ***region***).

Requête 9

Quelle est la surface moyenne des contrats à Paris ?

Code SQL :

```
SELECT ROUND (AVG (c.Surface),2) AS "Surface moyenne des contrats à Paris"
FROM contrat c
JOIN region r
ON r.Code_dep_code_commune =c.Code_dep_code_commune
WHERE r.dep_code = '75';
```

Résultat :

	Surface moyenne des contrats à Paris numeric
1	51.77

Description du code :

La fonction **SELECT** calcule la surface moyenne des contrats à Paris avec **AVG**, appliquée à la colonne [Surface] de la table ***contrat***. La colonne est renommée "Surface moyenne des contrats à Paris" avec **AS**, et **ROUND** arrondit le résultat à deux décimales. La fonction **JOIN** relie les tables ***contrat*** et ***region*** via la clé [Code_dep_code_commune]. La clause **WHERE** filtre les données pour inclure uniquement celles correspondant à Paris ([dep_code] = '75').

Requête 10

Classement des 10 départements où le prix moyen de la cotisation est le plus élevé.

Code SQL :

```
SELECT r.dep_nom, ROUND(AVG(c.Prix_cotisation_mensuel),2) AS "Prix moyen de cotisation
mensuel"
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune = r.Code_dep_code_commune
GROUP BY r.dep_nom
ORDER BY "Prix moyen de cotisation mensuel" DESC
LIMIT 10;
```

Résultat :

	dep_nom character varying	Prix moyen de cotisation mensuel numeric
1	Paris	36.40
2	Hauts-de-Seine	26.27
3	Val-de-Marne	19.82
4	Yvelines	18.89
5	Rhône	18.49
6	Ain	18.24
7	Alpes-Maritimes	18.14
8	Charente-Maritime	17.32
9	Haute-Savoie	17.15
10	Corse-du-Sud	17.07

Description du code :

La fonction **SELECT** extrait le nom des départements [dep_nom] de la table ***region*** et calcule le prix moyen des cotisations mensuelles avec **AVG**, appliqué à la colonne [Prix_cotisation_mensuel] de la table ***contrat***. **ROUND** arrondit ce prix moyen à deux décimales. **JOIN** relie les tables ***contrat*** et ***region*** via la clé commune [Code_dep_code_commune]. **GROUP BY** regroupe les données par département ([dep_nom]), et **ORDER BY** trie les résultats en ordre décroissant du prix moyen. **LIMIT** limite l'affichage aux 10 premiers départements avec les cotisations les plus élevées.

Requête 11

Liste des communes ayant eu au moins 150 contrats.

Code SQL :

```
SELECT r.com_nom, COUNT(*) AS nombre_de_contrats
FROM region r
JOIN contrat c
ON r.Code_dep_code_commune = c.Code_dep_code_commune
GROUP BY r.com_nom
HAVING COUNT(*) >= 150
ORDER BY nombre_de_contrats DESC;
```

Résultat :

	com_nom character varying	nombre_de_contrats bigint
1	PARIS 18	515
2	PARIS 17	468
3	PARIS 15	407
4	PARIS 16	394
5	NICE	387
6	PARIS 11	381
7	BORDEAUX	302
8	PARIS 20	302
9	NANTES	291
10	PARIS 19	266
11	PARIS 10	263
12	PARIS 12	252
13	PARIS 14	222
14	GRENOBLE	220
15	PARIS 9	204
16	TOULOUSE	187
17	TOULON	170
18	COURBEVOIE	163
19	LILLE	161
20	PARIS 3	159

Description du code :

La fonction **SELECT** extrait le nom des communes [com_nom] de la table ***region*** et le nombre total de contrats avec **COUNT** appliqué à la table ***contrat***. **JOIN** relie les tables via la clé [Code_dep_code_commune]. **GROUP BY** regroupe les données par communes, et **HAVING** filtre celles ayant au moins 150 contrats (COUNT(*) >= 150). **ORDER BY** trie les résultats par ordre décroissant du nombre de contrats (nombre_de_contrats).

Requête 12

Quel est le nombre de contrats pour chaque région ?

Code SQL :

```
SELECT r.reg_nom, COUNT(*) AS nombre_de_contrats
FROM contrat c
JOIN region r
ON c.Code_dep_code_commune = r.Code_dep_code_commune
GROUP BY r.reg_nom
ORDER BY nombre_de_contrats DESC;
```

Résultat :

	reg_nom character varying	nombre_de_contrats bigint
1	Ile-de-France	14177
2	Provence-Alpes-Côte d'Azur	3279
3	Auvergne-Rhône-Alpes	3042
4	Nouvelle-Aquitaine	2038
5	Occitanie	1609
6	Pays de la Loire	1196
7	Hauts-de-France	1189
8	Bretagne	947
9	Normandie	824
10	Grand Est	769
11	Centre-Val de Loire	598
12	Bourgogne-Franche-Comté	293
13	Corse	247
14	Martinique	73
15	Guyane	37
16	Nouvelle C	9
17	La Réunion	8

Description du code :

La fonction **SELECT** extrait le nom des régions [reg_nom] depuis la table ***region*** et calcule le nombre total de contrats avec **COUNT** appliqué à la table ***contrat***. **JOIN** relie les tables via la clé commune [Code_dep_code_commune]. **GROUP BY** regroupe les données par régions, et **ORDER BY** trie les résultats en ordre décroissant selon le nombre de contrats (nombre_de_contrats).